



SAPIENZA
UNIVERSITÀ DI ROMA

Motion Retargeting for Grasping Tasks

Mapping synergies from humans to robotic hands with dissimilar kinematics

A.A. 2024 - 2025

Based on the work of G. Gioioso, G. Salviati, M. Malvezzi, and D. Prattichizzo (2013), IEEE Transactions on Robotics[1]

Matteo Zamponi
Luca Colamarino
Giuseppe D'Addario
Federico Tranzocchi
Sapienza University of Rome

Contents

1	Introduction	3
1.1	The correspondence problem	3
1.2	Synergy-driven input	4
1.3	Objective	4
1.4	Structure	5
2	Background	6
2.1	Postural synergies	6
2.2	Standard motion retargeting strategies	7
2.3	Object-based retargeting	7
3	Framework	9
3.1	Reference points and sphere definition	9
3.2	Interaction matrix	10
3.3	Retargeting control law	10
3.4	Redundancy resolution	11
4	Experimental setup	13
4.1	System overview	13
4.2	Human input layer	13
4.3	Software implementation	14
4.4	Robot models	15
4.4.1	Barrett Hand	15
4.4.2	Mia Hand	16
4.4.3	Shadow Dexterous Hand	17
4.5	Evaluation metrics	18
4.5.1	Geometry metrics	18
4.5.2	Energy metrics	18
4.6	Qualitative validation	19
4.6.1	Simulation physics setup	19
5	Results	21
5.1	Quantitative analysis	21
5.1.1	Barrett Hand	22
5.1.2	Mia Hand	23
5.1.3	Shadow Dexterous Hand	24
5.2	Impact of redundancy resolution	25
5.3	Qualitative validation	25
5.3.1	Barrett Hand	26

5.3.2	Mia Hand	26
5.3.3	Shadow Hand	26
6	Conclusion	28
6.1	Summary	28
6.2	Future work	28

Chapter 1

Introduction

Robotic hands are becoming increasingly common in research and industry. Modern robotic hands often have many degrees of freedom (DoF) to imitate the dexterity of the human hand, which has over 20 DoF (e.g., Figure 1.1 shows a typical example of robotic hand (right)). A high number of DoF provides greater dexterity and the ability to carry out complex manipulation tasks. In fact, as humans, we can perform a huge variety of grasps in a natural way. However, controlling all of these DoF individually is generally not feasible for a robotic hand, as it would require an impractical number of actuators: robotic hand models are generally *underactuated*, meaning they have fewer actuators than DoF.

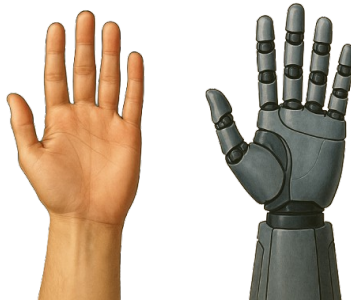


Figure 1.1: Comparison between a human hand (left) and a robotic hand (right).

This concept goes under the name of *postural synergies* [2]: the human brain controls the hand easily using coordinated patterns, the synergies, that reduce the complexity of hand control.

This project addresses the problem of *motion retargeting* from a human hand to a robotic hand, which can also be non-anthropomorphic. The goal is to leverage the patterns described by the postural synergies to control a target robotic hand that has a different kinematic structure from that of the human hand. To do this, we use an object-based mapping approach called the *virtual sphere* method [1], which focuses on the interaction between the hand and the object being manipulated, rather than on the specific structure of the hand itself.

1.1 The correspondence problem

A major challenge in teleoperation is the *correspondence problem*. Human hands and robotic hands are rarely identical: they usually have different bone lengths, different types of joints, and a different number of fingers. Because of these differences, standard mapping techniques like

mapping joint angles directly or mapping fingertip positions fail to accurately translate human motion. To solve this, we adopt an approach proposed by Gioioso et al. [1] that works in the *object domain*. Instead of mapping the anatomy of the hand directly, we map the effect that the hand has on a virtual object (a sphere) held in the hand, as illustrated in Figure 1.2.

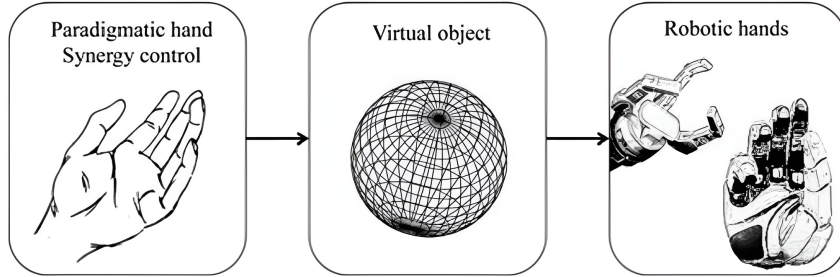


Figure 1.2: Schematic representation of the synergy-driven control of a robotic hand via the virtual sphere method. The human hand controls a virtual sphere, whose motion is then replicated by the robotic hand.

1.2 Synergy-driven input

To control the robotic hand, we first need a reliable reconstruction of the human hand pose. In this project, we use the Weart TouchDIVER G1 haptic glove in Figure 1.3, which provides the closure of three fingers (thumb, index, middle) and the abduction of the thumb, plus other measurements that are not of interest for this work.



Figure 1.3: The Weart TouchDIVER G1 haptic glove. (Image source: <https://weart.it/task-operations/>).

To reconstruct the full hand pose from this limited input, we rely on a reconstruction system developed in a previous work by Primiceri et al. [3]. Their system uses neural networks to estimate the full human hand pose from the sparse glove data, based on the theory of *postural synergies* [2]. We take this reconstructed hand pose as the input for our motion retargeting system, focusing on the mathematical translation of this motion to the robotic hand.

1.3 Objective

Our main goal is to implement and validate a robust pipeline that allows a human operator to control robotic hands with different kinematics in real-time. Specifically, we present an

algorithm that creates a virtual sphere inside the human hand and maps its deformation and movement to the robotic hand. Thanks to the mapping in the domain of the manipulated object, we create a generalized kinematic solver that can handle different robotic structures (e.g., from a 5-fingered hand to a 3-fingered one, or a 2-fingered gripper) without needing to redesign the mapping for each case.

Finally, we evaluate the performance of our system by analyzing the fidelity of the retargeting process. In particular, we assess how accurately the robotic hand replicates the deformation of the virtual sphere compared to the human reference, measuring the error in terms of sphere radius and position. Additionally, we analyze the energy associated with the elastic deformation of the grasp to quantify the similarity between the human and robotic grasping strategies.

1.4 Structure

The rest of this report is structured as follows. In Chapter 2 we review the relevant literature on hand synergies and motion retargeting techniques. In Chapter 3 we provide a mathematical formulation of the virtual sphere method and our implementation details. In Chapter 4 we describe the whole system architecture, including the hardware and software components, and the error metrics used for the simulation. In Chapter 5 we present the results of our experiments and evaluate the performance of the retargeting system. Finally, in Chapter 6 we summarize our findings and discuss potential future work.

Chapter 2

Background

In this chapter, we review the theoretical foundations and existing methodologies that form the basis of our work. We first discuss the biomechanical principles of human hand control, specifically the concept of postural synergies. We then analyze standard approaches for mapping human motion to robotic hands, highlighting their limitations when dealing with dissimilar kinematics. Finally, we introduce the object-based retargeting approach, which provides the theoretical framework for the virtual sphere method used in this project.

2.1 Postural synergies

The human hand is a kinematic structure with more than 20 degrees of freedom (DoF) controlled by a complex network of muscles and tendons. Despite this mechanical complexity, humans are able to grasp objects of different shapes and sizes with ease and dexterity. Neuroscientific studies suggest that the central nervous system simplifies the control of the hand by coordinating the movement of multiple joints through a reduced set of control variables known as *postural synergies* [2]. These synergies represent patterns of joint coordination that capture the most significant variations in hand posture during grasping tasks.

In their study, Santello et al. [2] analyzed the hand postures of different subjects while grasping imaginary objects. By applying *Principal Component Analysis* (PCA) to the collected joint angle data, they identified that a large portion of the variance in hand postures could be explained by a very small number of principal components. Specifically, the first principal components (*synergies*) account for more than 80% of the variance in hand posture: the first synergy corresponds to the coordinated flexion and extension of all fingers (opening and closing the hand), resembling a power grasp (Figure 2.1, horizontal axis); the second synergy accounts for the abduction of the fingers and the opposition of the thumb, effectively controlling the arching of the palm (Figure 2.1, vertical axis). The remaining synergies can be used to fine-tune the hand posture for specific grasp types, but contribute increasingly less to the overall variance. We graphically illustrate the underlying flow of information in human hand control with the block diagram in Figure 2.2.

This finding is fundamental to our work: it implies that we do not need to measure every single joint of the human hand to understand its pose. Instead, we can capture the underlying correlations to reconstruct the full hand posture from the sparse data provided by the Weart glove, as demonstrated by the neural network approach [3] that we use.

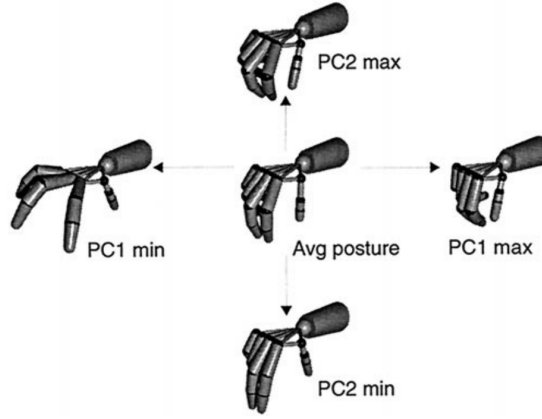


Figure 2.1: Visual representation of the first two postural synergies. Synergy 1 (on the horizontal axis) controls the general opening and closing of the hand. Synergy 2 (on the vertical axis) modulates the arching of the palm and finger adduction/abduction.

2.2 Standard motion retargeting strategies

Retargeting human hand motion to a robotic hand is a classic problem in robotics, often referred to as the *correspondence problem*. The challenge arises from the fact that robotic hands typically have different kinematic structures, sizes, and joint limits compared to the human hand. In literature, we can find two main strategies to address this problem.

Joint-to-joint mapping This is the most direct approach, where the joint angles of the human hand are mapped one-to-one to the corresponding joints of the robotic hand. If the robot is anthropomorphic, we can map human joints $\mathbf{q}_h$ directly to robot joints $\mathbf{q}_r$ using a linear transformation; if the robot has fewer joints, we need a mapping function to approximate the motion.

This method is straightforward and relatively easy to implement, but it fails with non-anthropomorphic hands (e.g., a 3-fingered gripper). Since the kinematic chains differ, applying human angles directly can lead to unnatural or infeasible robot postures, self-collisions, or the inability to grasp objects properly.

Fingertip (Cartesian) mapping To overcome the limitations of joint-to-joint mapping, another common approach is to focus on the positions of the fingertips rather than the joint angles. The Cartesian coordinates of the human fingertips $\mathbf{p}_h$ are computed through forward kinematics and then used as target positions for the robot's fingertips $\mathbf{p}_r$. An inverse kinematics solver is then employed to find the joint angles $\mathbf{q}_r$ that achieve these positions.

While this ensures that the fingertips reach the target, it completely ignores the internal configuration of the hand, potentially leading to unnatural grasps or excessive joint movements. Additionally, if the robot has fewer fingers than the human hand, it becomes unclear how to map multiple human fingertips to fewer robotic ones: a fingertip position reachable by a human might be a singular or unreachable configuration for the robot.

2.3 Object-based retargeting

To address the limitations of the standard retargeting methods, Gioioso et al. [1] proposed an approach defined in the *object domain*: instead of mapping the hand itself, either in joint or

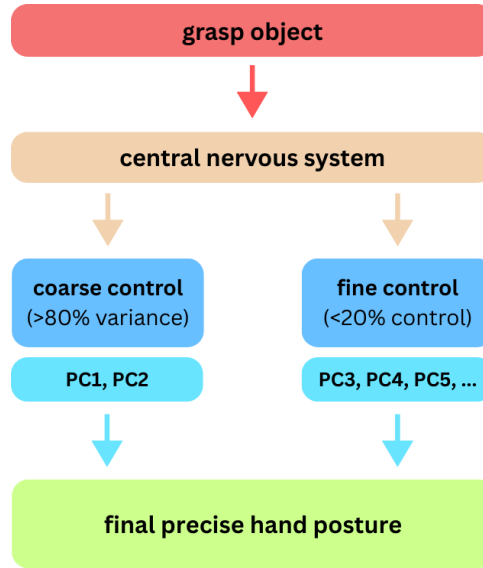


Figure 2.2: The hierarchical control scheme of the hand. Coarse control is achieved via the first two synergies ($> 80\%$ variance), while fine control is handled by higher-order synergies ($< 20\%$ variance).

Cartesian space, we map the effect the hand has on the object being manipulated. The key idea is that the primary goal of the hand is to interact with objects, so by focusing on the object, we can achieve more natural and effective grasps.

Since the object might not physically exist during teleoperation (e.g., in virtual reality scenarios), a *virtual object* (a sphere) is introduced in the grasping process. The method proceeds in the following steps:

1. A virtual sphere is mathematically fitted inside the human hand, defined by a set of reference points (e.g., fingertips and palm center). As the human hand moves via synergies, this sphere *translates*, *rotates*, and *deforms* accordingly.
2. The motion of the human sphere is scaled to match the size of the robotic hand, allowing a large human hand to control a small robotic gripper, or vice versa.
3. The robot is commanded to move its joints such that its own virtual sphere mimics the transformation of the human's sphere.

This object-based retargeting method is *independent* of the kinematic structure of both hands. It captures the intention of the grasp, namely whether the user is squeezing (shrinking the sphere) or moving the hand (translating the sphere), and it applies to the robot regardless of its number of fingers or joint structure.

Chapter 3

Framework

In this chapter, we present the mathematical formulation of the proposed motion retargeting framework. We detail the definition of the virtual sphere, the construction of the interaction matrix that relates hand motion to object deformation, and the derivation of the control law used to drive the robotic hand. Finally, we describe the redundancy resolution strategy employed to optimize the configuration of the robotic hand during manipulation task.

3.1 Reference points and sphere definition

The core concept of the object-based mapping is to abstract the hand's motion into the motion of a virtual object. We model this object as a *virtual sphere* defined by a set of reference points on the hand (see Figure 3.1 for an example). These reference points can include fingertips, the palm center, or any other significant point that captures the motion of the hand while grasping.

Let $\mathbf{p}_h \in \mathbb{R}^{3N_h}$ be the vector of the Cartesian positions of N_h reference points on the human hand. Similarly, let $\mathbf{p}_r \in \mathbb{R}^{3N_r}$ be the vector of N_r reference points on the robotic hand. At any time step t , the virtual sphere is defined as the *minimum enclosing ball*, i.e., the sphere of smallest radius that contains all reference points. Such a sphere is characterized by its center $\mathbf{o} \in \mathbb{R}^3$ and radius $r \in \mathbb{R}$.

The objective of the retargeting algorithm is to impose that the virtual sphere of the robotic hand mimics the rigid-body motion (translation and rotation) and the non-rigid deformation (scaling) of the human virtual sphere. To account for the size difference between the two hands, we introduce a scaling factor k_{sc} :

$$k_{sc} = \frac{r_r}{r_h} \quad (3.1)$$

where r_r and r_h are the radii of the robotic and human virtual spheres in their initial reference configurations, respectively.

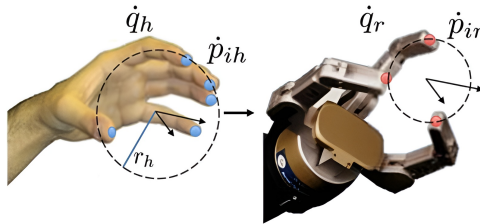


Figure 3.1: Example of virtual spheres defined by reference points on the human (left, blue dots) and robotic (right, red dots) hands.

3.2 Interaction matrix

To map the motion of the reference points to the motion of the sphere, we use the definition provided by Gioioso et al. [1]. The velocity of a generic reference points $\mathbf{p}_i$ can be expressed as a function of the sphere's linear velocity $\dot{\mathbf{o}}$, angular velocity $\boldsymbol{\omega}$, and rate of change of radius $\dot{r}$:

$$\dot{\mathbf{p}}_i = \dot{\mathbf{o}} + \boldsymbol{\omega} \times (\mathbf{p}_i - \mathbf{o}) + \dot{r}(\mathbf{p}_i - \mathbf{o}) \quad (3.2)$$

By stacking the velocities of all reference points, we can express the relationship between the sphere motion and the reference points' motion in matrix form:

$$\dot{\mathbf{p}} = \mathbf{A} \mathbf{v}_{\text{obj}} \quad (3.3)$$

where $\mathbf{v}_{\text{obj}} = [\dot{\mathbf{o}}^T, \boldsymbol{\omega}^T, \dot{r}]^T \in \mathbb{R}^7$ represents the generalized velocity of the virtual object. The matrix $\mathbf{A} \in \mathbb{R}^{3N \times 7}$ is the *interaction matrix*, constructed as follows:

$$\mathbf{A} = \begin{bmatrix} \mathbf{I}_3 & -[\mathbf{p}_1 - \mathbf{o}]_{\times} & (\mathbf{p}_1 - \mathbf{o}) \\ \vdots & \vdots & \vdots \\ \mathbf{I}_3 & -[\mathbf{p}_N - \mathbf{o}]_{\times} & (\mathbf{p}_N - \mathbf{o}) \end{bmatrix} \quad (3.4)$$

Here, $\mathbf{I}_3$ is the 3×3 identity matrix, and $[\cdot]_{\times}$ denotes the skew-symmetric matrix operator for the cross product. This matrix relates the task-space velocities of the reference points to the deformation of the object.

3.3 Retargeting control law

Our retargeting strategy relies on mapping the velocity of the virtual object from the human domain to the robot domain. In the original formulation by Gioioso et al. [1], the human hand velocity is derived analytically from the synergy inputs.

Let $\mathbf{z} \in \mathbb{R}^{n_z}$ be the vector of synergy activation coefficients. The human joint velocities $\dot{\mathbf{q}}_h$ can be expressed as:

$$\dot{\mathbf{q}}_h = \mathbf{S} \dot{\mathbf{z}} \quad (3.5)$$

where $\mathbf{S} \in \mathbb{R}^{n_{q_h} \times n_z}$ is the *synergy matrix* mapping low-dimensional inputs to the full joint space. Consequently, the velocity of the human reference points $\dot{\mathbf{p}}_h$ would be computed as:

$$\dot{\mathbf{p}}_h = \mathbf{J}_h \dot{\mathbf{q}}_h = \mathbf{J}_h \mathbf{S} \dot{\mathbf{z}} \quad (3.6)$$

where $\mathbf{J}_h \in \mathbb{R}^{3N_h \times n_{q_h}}$ is the human hand Jacobian.

However, in our specific architecture, the reconstruction of the human hand pose is performed by a neural network [3], which directly outputs the full joint configuration $\mathbf{q}_h$ of the virtual human hand in Unity. Since the full pose is known at every frame, computing the analytical Jacobian $\mathbf{J}_h$ and the synergy matrix $\mathbf{S}$ explicitly is unnecessary. Instead, we obtain the reference point velocities $\dot{\mathbf{p}}_h$ via *numerical differentiation* of the tracked points in the virtual environment:

$$\dot{\mathbf{p}}_h(t) \approx \frac{\mathbf{p}_h(t) - \mathbf{p}_h(t - \Delta t)}{\Delta t} \quad (3.7)$$

This approach allows us to bypass the complexity of modeling the human kinematic chain analytically while ensuring that the motion fed into the retargeting algorithm accurately reflects the reconstructed hand pose.

Once $\dot{\mathbf{p}}_h$ is obtained, we compute the generalized velocity $\mathbf{v}_{\text{obj},h}$ of the human virtual sphere by inverting the interaction matrix. Since $\mathbf{A}_h$ is typically tall (more reference points than object DoFs), we use the *Moore-Penrose pseudoinverse* $\mathbf{A}_h^\#$:

$$\mathbf{v}_{\text{obj},h} = \mathbf{A}_h^\# \dot{\mathbf{p}}_h \quad (3.8)$$

Next, we map this motion to the robotic domain using a scaling matrix $\mathbf{K}_c \in \mathbb{R}^{7 \times 7}$:

$$\mathbf{v}_{\text{obj},r} = \mathbf{K}_c \mathbf{v}_{\text{obj},h} \quad (3.9)$$

The matrix $\mathbf{K}_c$ scales the translation and radial growth components of the object velocity by the factor k_{sc} , while leaving the angular velocity unchanged (which is independent of size):

$$\mathbf{K}_c = \begin{bmatrix} k_{sc} \mathbf{I}_3 & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_3 & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & \mathbf{0}_{1 \times 3} & k_{sc} \end{bmatrix} \quad (3.10)$$

We then compute the target velocity for the robot reference points $\dot{\mathbf{p}}_{r,\text{des}} = \mathbf{A}_r \mathbf{v}_{\text{obj},r}$. To track these points, we use the robot's differential kinematics equation $\dot{\mathbf{p}}_r = \mathbf{J}_r \dot{\mathbf{q}}_r$, where $\mathbf{J}_r$ is the Jacobian matrix mapping robot joint velocities $\dot{\mathbf{q}}_r$ to end-effector velocities $\dot{\mathbf{p}}_r$.

The final control law for the robot joint velocities is obtained by inverting this equation:

$$\dot{\mathbf{q}}_r = \mathbf{J}_{r,\text{DLS}}^\# \dot{\mathbf{p}}_{r,\text{des}} \quad (3.11)$$

where $\mathbf{J}_r^\#$ is the *Damped Least Squares* (DLS) inverse of the robot Jacobian, which ensures numerical stability even when the robot is near singular configurations.

3.4 Redundancy resolution

In those scenarios where particularly dexterous robotic hands are employed, one can exploit their kinematic redundancy to optimize their internal configuration while performing grasping tasks.

To achieve this, we can augment the control law derived in Eq. 3.11 with a secondary objective using the *null-space projection* method: we exploit the null space of the Jacobian to perform secondary tasks without affecting the primary goal of tracking the virtual sphere motion. The modified control law becomes:

$$\dot{\mathbf{q}}_r = \underbrace{\mathbf{J}_{r,\text{DLS}}^\# \dot{\mathbf{p}}_{r,\text{des}}}_{\dot{\mathbf{q}}_{r,\text{primary}}} + (\mathbf{I} - \mathbf{J}_{r,\text{DLS}}^\# \mathbf{J}_r) \dot{\mathbf{q}}_0 \quad (3.12)$$

The term $(\mathbf{I} - \mathbf{J}_{r,\text{DLS}}^\# \mathbf{J}_r)$ projects an *arbitrary* velocity vector $\dot{\mathbf{q}}_0$ into the null space of the primary task. This allows us to define $\dot{\mathbf{q}}_0$ as the gradient of a performance criterion $H(\mathbf{q}_r)$ designed to keep the joints of the robotic hand away from the mechanical limits. Following standard kinematic control theory, we utilize the *joint range* availability function:

$$H(\mathbf{q}_r) = \frac{1}{2N} \sum_{i=1}^N \left(\frac{q_i - \bar{q}_i}{q_{i,\text{max}} - q_{i,\text{min}}} \right)^2 \quad (3.13)$$

where N is the number of joints, $q_{i,\text{max}}$ and $q_{i,\text{min}}$ are the upper and lower limits, and $\bar{q}_i$ is the midpoint of the range of joint i .

The secondary velocity task is defined as the steepest descent direction of this function:

$$\dot{\mathbf{q}}_0 = -\eta \nabla_{\mathbf{q}} H \quad (3.14)$$

where η is a positive scalar gain that regulates the influence of the secondary task, and the i -th component of the gradient is:

$$\frac{\partial H}{\partial q_i} = \frac{1}{N} \frac{q_i - \bar{q}_i}{(q_{i,\max} - q_{i,\min})^2} \quad (3.15)$$

This formulation normalizes the error, ensuring that joints with smaller ranges are prioritized (pushed harder towards the center of their range) compared to joints with larger ranges.

For underactuated grippers with fewer degrees of freedom than the task requirements, the null space term naturally vanishes or has no effect. In these cases, the pseudoinverse in Eq. 3.11 provides the least-squares solution that minimizes the error between the desired and actual sphere motion.

Chapter 4

Experimental setup

In this chapter, we describe the complete hardware and software architecture developed to validate the proposed retargeting framework. We will first go through the data flow from the input device to the simulation environment, delve into the software implementation of the control algorithm, present the specific robotic hand models used in our experiments, and finally define the metrics employed to evaluate the performance of the whole architecture.

4.1 System overview

The motion retargeting pipeline shown in Figure 4.1 is designed as a modular system composed of three main blocks: the *input layer* (haptic glove and reconstruction server), the *retargeting layer* (unity simulation and retargeting logic), and the *output layer* (robotic hand models).

The data flow consists of the following steps:

1. *Acquisition*: the user wears the Weart TouchDIVER G1 haptic glove, which captures sensor data (finger closure and abduction) that are sent to a Python server.
2. *Reconstruction*: the Python server processes the glove inputs using a neural network [3] to reconstruct the full pose of the virtual human hand in Unity.
3. *Transmission*: the reconstructed joint angles are sent to the Unity simulation environment in real-time.
4. *Retargeting*: inside Unity, the retargeting algorithm detailed in Chapter 3 computes the deformation of the virtual sphere and derives the corresponding joint velocities for the target robotic hand model.
5. *Actuation*: the resulting joint velocities are integrated to update the visual pose of the robotic hand in the simulation.

4.2 Human input layer

The input interface is the Weart TouchDIVER G1, a wearable haptic device capable of tracking hand movements and rendering force, texture, and thermal cues. For the scope of this work, we focus on its motion tracking capabilities: it provides raw data about the closure of the thumb, index, and middle fingers, as well as the abduction of the thumb relative to the palm.

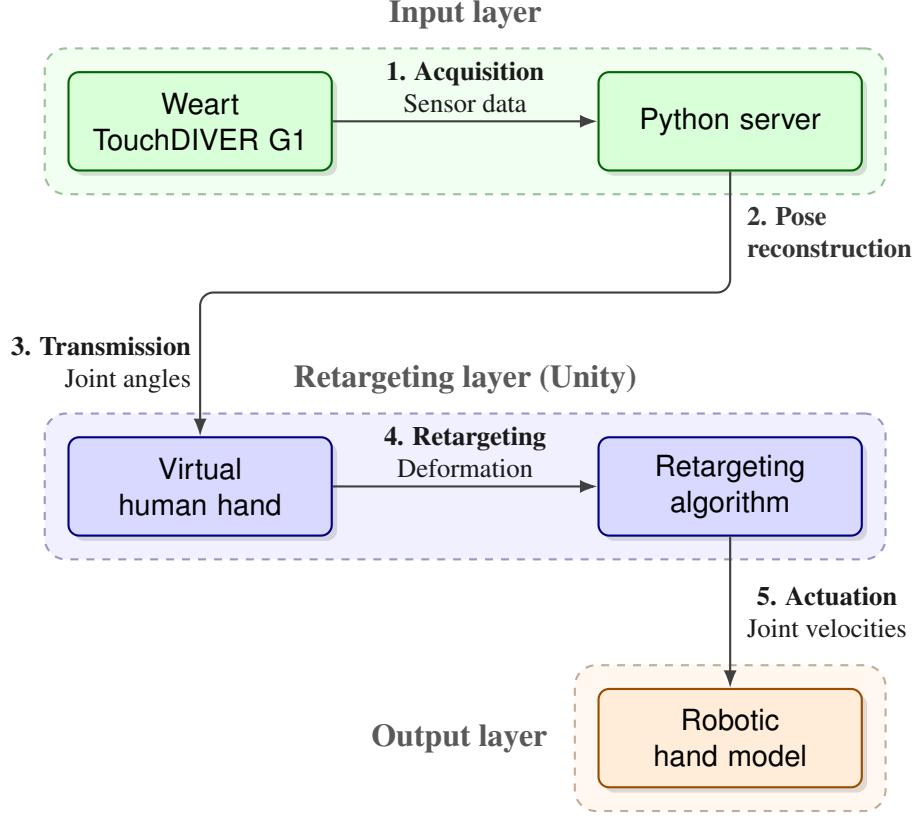


Figure 4.1: System architecture overview showing the complete motion retargeting pipeline. The data flows from the haptic glove through the reconstruction server, into the Unity-based retargeting layer, and finally to the robotic hand model for actuation.

Since the glove does not track the ring and pinky fingers, nor the individual phalanx rotations (MCP, PIP, DIP) explicitly, the raw sensor data is insufficient for teleoperation. To address this limitation, we employ the reconstruction module developed by Primiceri et al. [3]: a Python script acts as a server listening for incoming data from the glove, and feeds this data into a pre-trained neural network which outputs a 62-dimensional vector containing the sine and cosine encodings of all 15 human hand joints; this vector is then converted back into Euler angles and sent to the Unity client.

4.3 Software implementation

The core logic of the project is implemented in C# in the Unity engine. The implementation relies on the `MathNet.Numerics` library for efficient linear algebra computations, such as matrix multiplications and pseudoinverses.

Communication A dedicated thread handles the TCP communication with the Python server to separate the network operations from the main Unity thread, so as to not interfere with the simulation’s frame rate. The received human pose $\mathbf{q}_h$ is applied to a kinematic model of the human hand, which serves as the *master* for the retargeting algorithm.

Kinematic solver To guarantee the system is adaptable to different robots, we developed a generalized `Kinematics` library: unlike standard solutions that hardcode the Jacobian matrix

for a specific robot, our implementation computes the robot Jacobian $\mathbf{J}_r$ dynamically. We define the robot structure in the Unity inspector by assigning a list of joint Transforms and their corresponding types (e.g., `HingeX`, `HingeY`, `HingeZ`, or `Ball`), allowing the software to handle any serial kinematic chain without code modification.

Virtual sphere The `VirtualSphere` script implements the mathematical framework described in Chapter 3 for the deformation and retargeting logic. It takes as input an array of `Transform` objects representing the reference points, and at every update:

1. it extracts the local positions of the reference points with respect to the palm;
2. it computes the *minimum enclosing ball* using Welzl’s algorithm;
3. it constructs the interaction matrix $\mathbf{A}$ from Eq. 3.4 based on the sphere’s current center and radius.

This script is attached to both the human hand and the robotic hand, providing the necessary matrices $\mathbf{A}_h$ and $\mathbf{A}_r$ to the main controller.

Code The complete source code developed for this project is open source and available on GitHub.¹

4.4 Robot models

To evaluate the flexibility of the object-based retargeting approach [1], we selected robotic hands with significantly different kinematic structures to that of the human hand. In our experiments, we used three specific models: the *Barrett Hand*, the *Mia Hand*, and the *Shadow Dexterous Hand*. The kinematic descriptions (URDF models) for the Barrett and Mia hands were obtained from the embodiment framework by Fabisch et al. [4], while the Shadow Hand model was sourced from the Bunny-VisionPro framework [5]. All of them provide a full implementation of the physical properties (dynamics, collisions, limits, etc.) for realistic experiments, and were imported into Unity using the *URDF Importer* package.

4.4.1 Barrett Hand

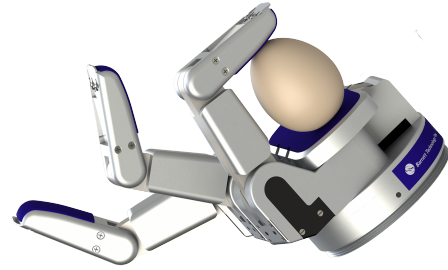
The *Barrett Hand* (Figure 4.2) is a multi-fingered programmable grasper whose kinematic structure is deeply different from the human hand, making it an interesting candidate for testing the robustness of the virtual sphere retargeting method.

Kinematics It consists of three fingers: one finger is fixed, while the other two can rotate synchronously around the palm (*spread motion*) up to 180°. This allows the hand to change its configuration dynamically, switching from a parallel grasp with the three fingers aligned, to an opposition grasp with the two rotating fingers opposing the fixed one.

¹https://github.com/yvhem/object_domain_synergy_mapping



(a) Kinematic structure



(b) Grasping an egg

Figure 4.2: The Barrett Hand. (a) shows the hand in an open configuration, highlighting the three-finger design (Source: ROS.org). (b) demonstrates the adaptability of the hand grasping an egg (Source: Barrett Technology).

Actuation The hand has 8 axes of motion but is *underactuated*, driven by only 4 motors: one controls the spread of the two movable fingers, and the other three control the flexion of each finger independently.² A proprietary "TorqueSwitch" mechanism couples the proximal and distal links: the distal link remains stationary until the proximal link encounters resistance, at which point the distal link curls to enclose the object. However, we are interested in evaluating the retargeting performance based on kinematics alone; therefore, in our kinematic model of the Barrett Hand, we treat all 8 axes as independent degrees of freedom.

4.4.2 Mia Hand

The *Mia Hand* (Figure 4.3) by Prensilia is an anthropomorphic end-effector designed primarily for prosthetics and research applications. Unlike the Barrett Hand, the Mia Hand has a kinematic structure that resembles that of the human hand, although with less dexterity.



(a) Anthropomorphic structure



(b) Cylindrical grasp

Figure 4.3: The Mia Hand. (a) shows the hand's resting posture, highlighting the mechanical design and soft pads. (b) shows the hand performing a cylindrical grasp, demonstrating finger coordination. Source: Prensilia.

²BarrettHand BH8-282 specs: <https://barrett.com/barretthand>

Kinematics The Mia Hand's dimensions are similar to an average human hand (palm width 83 mm). It consists of five fingers and an opposable thumb, designed to perform various grasp types such as cylindrical, spherical, and lateral grasps.³

Actuation To keep a lightweight profile (approx. 540 g), the Mia Hand employs an under-actuated mechanism driven by 3 motors actuating the flexion/extension of the fingers and the opposition of the thumb. Again, like for the Barrett Hand, in our simulation we treat the hand's kinematic chain as fully articulated, meaning we control the individual joints of the fingers independently.

4.4.3 Shadow Dexterous Hand

The *Shadow Dexterous Hand* (Figure 4.4) is one of the most advanced anthropomorphic end-effectors available on the market, designed to reproduce the kinematics and dexterity of the human hand as closely as possible.

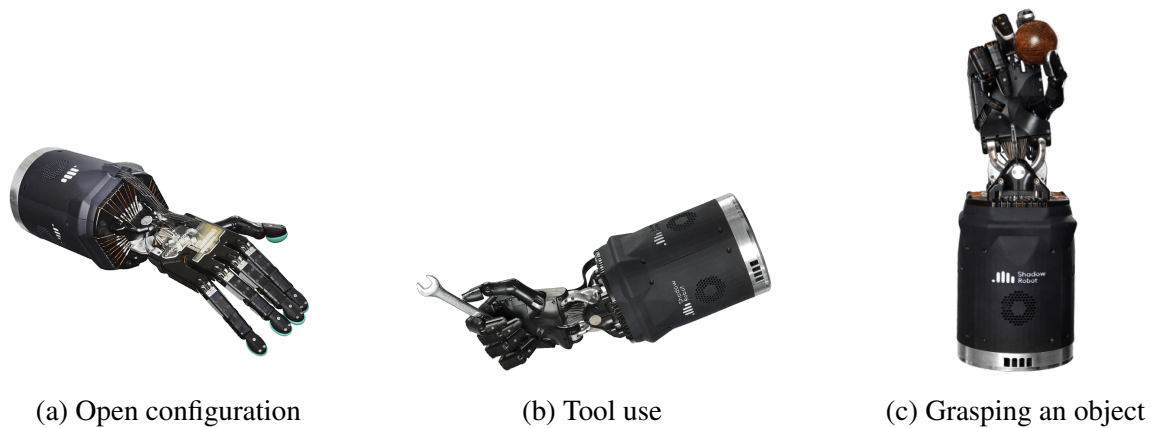


Figure 4.4: The Shadow Dexterous Hand. Its 24 degrees of freedom allow it to perform complex manipulation tasks, from power grasps to precision tool use. Source: Shadow Robot.

Kinematics The Shadow Hand has 24 joints (actuated and passive) providing 20 independent degrees of freedom: the thumb has 5 joints and 5 DoF, allowing for complex opposition movements, while each of the four fingers has 4 joints but only 3 DoF due to the mechanical coupling of the distal joint (DIP) and the middle joint (PIP). This design ensures that the angle of the middle joint is always greater than or equal to that of the distal joint, mimicking the natural behavior of human fingers. The little finger has an extra joint in the palm to allow for opposition (forming a palm arch). The wrist provides 2 DoF.

Actuation Unlike the significantly underactuated Barrett and Mia hands, the Shadow Hand is highly actuated: indeed, it has 24 joints driven by 20 electric "Smart Motors" located in the forearm base, which transmit force to the joints via tendons.⁴ This high number of degrees of freedom makes the Shadow Hand highly *kinematically redundant* with respect to the 7-DoF virtual sphere task. Therefore, the redundancy resolution strategy defined in Eq. 3.12 becomes

³Technical specification of the Mia Hand can be found at: <https://www.mia-hand.com/>

⁴Shadow Dexterous Hand technical specification: <https://shadowrobot.com/>

crucial to stabilize the internal configuration of the hand keeping the 20 actuated joints away from their mechanical limits, while the virtual sphere accurately defines the overall grasp shape.

4.5 Evaluation metrics

To quantitatively assess the performance of the retargeting algorithm, we define a set of metrics that measure how well the robotic hand tracks the desired object deformation and how effectively it uses its kinematic capabilities. Since the virtual sphere represents the core of our control strategy, these metrics focus on comparing the state of the virtual sphere of the robot against that of the human over time.

We define t_0 as the start time of the experiment. The subscripts H and R denote the Human (reference) and Robot (measured) quantities, respectively.

4.5.1 Geometry metrics

These metrics evaluate how accurately the robotic hand tracks the translation and radial deformation of the virtual object.

Radius error This metric measures the ability of the system to replicate the scaling of the sphere, namely opening and closing of the hand. Since the robot and human hands have different sizes, the radius of the human sphere r_H must be scaled by a factor k_{sc} (defined in Eq. 3.1) before comparison. The absolute radius error $e_{rad}(t)$ is defined as:

$$e_{rad}(t) = |r_R(t) - k_{sc}r_H(t)| \quad (4.1)$$

Position variation error This metric assesses the "drift" of the object within the hand. Ideally, if the human hand translates the sphere by a certain vector, the robot should replicate that same displacement vector, scaled according to the workspace dimensions defined by the hand size ratio k_{sc} . We therefore compare the displacement relative to the initial position at t_0 .

Let $\mathbf{p}(t) = [x, y, z]^T$ be the position of the sphere center. The relative displacement is defined as:

$$\Delta\mathbf{p}(t) = \mathbf{p}(t) - \mathbf{p}(t_0)$$

The position variation error $e_{pos}(t)$ is then obtained as the magnitude of the difference between the scaled human displacement and the robot displacement:

$$e_{pos}(t) = ||k_{sc}\Delta\mathbf{p}_H(t) - \Delta\mathbf{p}_R(t)|| \quad (4.2)$$

4.5.2 Energy metrics

Besides the geometry of the virtual sphere, we are also interested in evaluating the *intensity* of the grasp. In the virtual sphere framework, the deformation of the object corresponds to an accumulation of elastic potential energy in the virtual stiffness model.

Elastic energy error Let r_t and $r(t_0)$ be the virtual sphere radius at time t and the initial radius, respectively. The normalized deformation $\Delta(t)$ is computed as the absolute relative change in radius. For the human and robot spheres, this is defined respectively as:

$$\Delta_H(t) = \frac{|r_H(t) - r_H(t_0)|}{r_H(t_0)}, \quad \Delta_R(t) = \frac{|r_R(t) - r_R(t_0)|}{r_R(t_0)} \quad (4.3)$$

Unlike the radius error, this metric considers the normalized strain on the object regardless of its absolute size.

To compare the energy stored by hands with different numbers of fingers, and thus different numbers of reference points on the sphere, we introduce a contact point ratio $k_{cp} = N_H/N_R$, where N_H and N_R are the number of reference points for the human and the robot, respectively.

The elastic potential energy for the human (U_H) and the normalized energy for the robot (U_R) are computed as:

$$U_H(t) = \frac{1}{2}k_s\Delta_H^2(t), \quad U_R(t) = \frac{1}{2}k_{cp}k_s\Delta_R^2(t) \quad (4.4)$$

where k_s is the spring stiffness constant. The absolute energy error is finally defined as:

$$e_{\text{energy}}(t) = |U_R(t) - U_H(t)| \quad (4.5)$$

4.6 Qualitative validation

The quantitative metrics defined above provide a measure of kinematic accuracy, but they do not capture the functional success of a grasp in a practical scenario. To assess the applicability of the retargeting algorithm in real-world manipulation tasks, we also perform qualitative evaluations by simulating various grasping tasks within the Unity environment.

We selected three objects of standard geometric shapes to test different grasp types:

- *Sphere*: to test spherical grasps and the ability to conform to curved surfaces.
- *Cube*: to evaluate the ability of the robotic fingers to adapt to flat surfaces and edges.
- *Cylinder*: to assess the performance in cylindrical grasps, which are common in everyday tasks.

For each object, we simulate a grasping task using the Weart glove as the input device. We visually inspect the resulting robotic motion to determine two key aspects:

- Whether the robot fingers enclose around the object in a natural way without unfeasible interpenetration or unnatural internal joint configurations;
- whether the virtual object remains stably within the robotic hand during the closure phase.

4.6.1 Simulation physics setup

Simulating contact interactions between rigid bodies in Unity is complex, and a wrong configuration can lead to unrealistic behaviors such as the two models flying away from each other.

To mitigate this issue and simulate a realistic interaction, we introduced linear and angular *damping* to the manipulated objects. This mimics the effect of *friction* between the hand and the object, preventing this latter from reacting too abruptly to sudden contact forces.

Furthermore, we disabled gravity for the target objects. Since our experimental setup focuses strictly on the *grasping* phase, we skip the *approach* phase (where the hand moves towards the object) and fix the base of the robotic hand in space. Disabling gravity allows the object to remain suspended in the "grasping workspace", ensuring that the evaluation focuses solely on the closure of the fingers and the synergy mapping, rather than the robot's ability to chase a falling object.

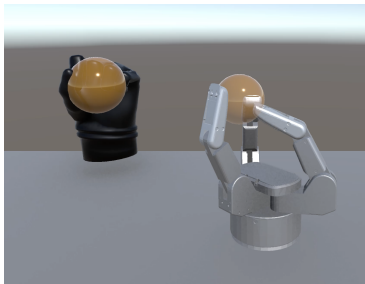
Finally, it is important to note that the outcome of these tests is heavily influenced by the quality of the *colliders* of the robotic models: inaccurate or simplified collision meshes, such as box colliders, can lead to missed contacts or interpenetrations, affecting the realism of the grasp.

Chapter 5

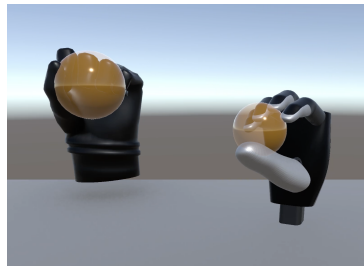
Results

In this chapter, we present the experimental results obtained from the validation of the motion retargeting framework. The analysis is divided into two parts: a quantitative evaluation based on the metrics defined in Chapter 4, and a qualitative assessment of the grasping capabilities on standard geometric objects.

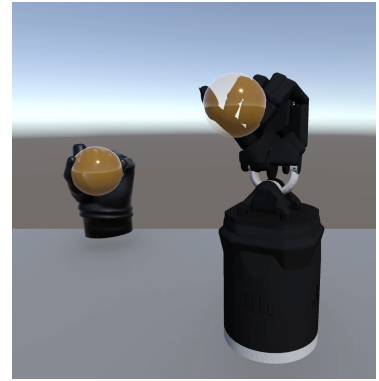
The experiments were performed using the three robotic hands described in the previous chapter: the Barrett Hand, the Mia Hand, and the Shadow Dexterous Hand. For the quantitative analysis, we recorded data during a standardized grasping task where the user closes their hand (shrinking the virtual sphere, see Figure 5.1) and translates it within the workspace. To isolate the performance of the retargeting algorithm from potential noise, these specific plots were generated using keyboard teleoperation to simulate a nominal closing motion. The qualitative validation, instead, uses the data recorded from the Weart Glove during natural human grasps.



(a) Virtual sphere representation for the Barrett Hand.



(b) Virtual sphere representation for the Mia Hand.



(c) Virtual sphere representation for the Shadow Dexterous Hand.

Figure 5.1: The three robotic hands (on the right) "holding" the virtual sphere compared to the human hand (on the left).

5.1 Quantitative analysis

We analyze the performance of the system in terms of geometric fidelity (radius and position tracking) and energetic fidelity (elastic energy estimation).

5.1.1 Barrett Hand

The Barrett Hand represents a significant test case due to its non-anthropomorphic structure and low number of actuated degrees of freedom (4 motors).

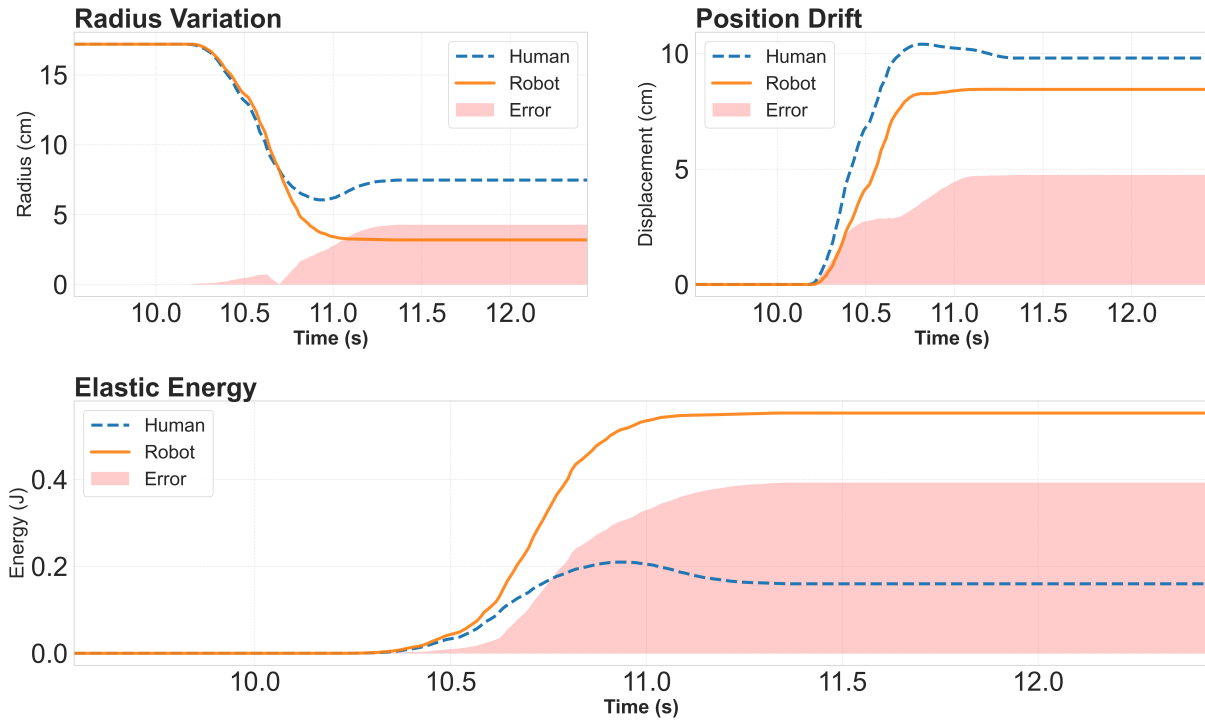


Figure 5.2: Performance metrics for the Barrett Hand.

Radius comparison As shown in the top-left graph of Figure 5.2, the retargeting algorithm successfully maps the closing motion. The grasp action occurs roughly between $t = 10.3$ s and $t = 11.0$ s. The human hand (blue dashed line) reduces the virtual sphere radius from ≈ 17 cm to a steady-state value of ≈ 7.5 cm, and the Barrett Hand (orange solid line) tracks this descent almost perfectly during the transient phase.

However, we observe that the robot achieves a significantly smaller final radius (≈ 3.5 cm) compared to the human hand. This behavior is expected due to the different kinematic structures: the Barrett Hand is capable of fully closing its fingers (see Chapter 4), whereas the human hand cannot completely close around a sphere due to anatomical constraints. This difference in the final posture results in a growing error (red area) during the steady-state phase.

Position variation The top-right graph illustrates the tracking of the hand position drift. The robot accurately follows the reference given by the human movement; in fact, the human hand translates its virtual sphere by ≈ 10 cm, and the robot reaches a displacement of ≈ 8.5 cm. This slight undershoot in the final position may be due to the physical parameters (stiffness, damping, force limit) we set for the imported URDF model of the Barrett Hand in the simulation environment, which were chosen in order to achieve stable grasps. Nonetheless, both the direction and timing of the reference motion are well replicated.

Elastic energy The bottom graph shows the elastic potential energy stored in the grasp, representing grasp intensity. Both the human and robot energy profiles begin to rise simultaneously

at around $t \approx 10.4$ s, indicating good synchronization during the initial phase of the grasp.

The most notable feature is the difference in magnitude at steady state: the robot stabilizes at a much higher energy level, specifically more than three times that of the human hand (≈ 0.55 J vs. ≈ 0.16 J). This is a direct mathematical consequence of the radius tracking observed earlier. Since the energy metric is proportional to the square of the normalized deformation, and the robot achieved a much tighter grasp (smaller final radius) because of the capabilities of the Barrett, the calculated energy is significantly higher. This indicates that the retargeted motion resulted in a very firm and secure grasp.

5.1.2 Mia Hand

The Mia Hand shares the underactuated nature of the Barrett Hand but features an anthropomorphic design, as we discussed in Chapter 4.

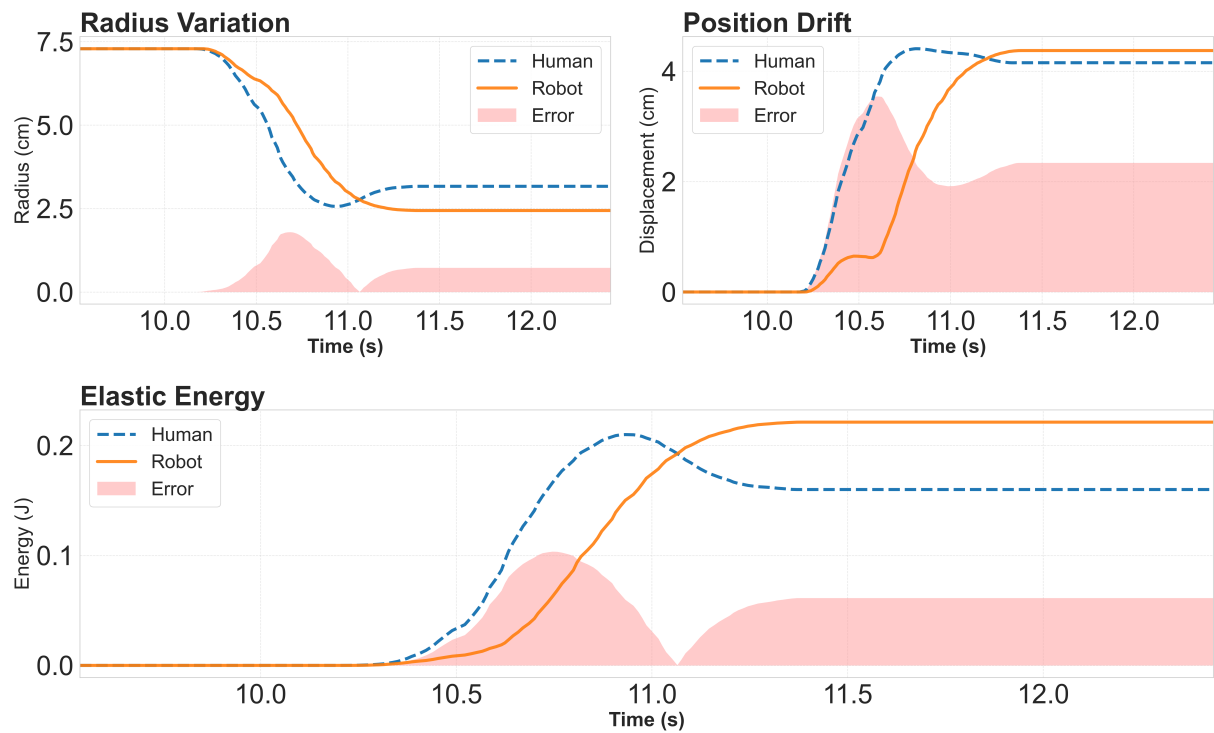


Figure 5.3: Performance metrics for the Mia Hand.

Radius comparison From the top-left graph in Figure 5.3, we observe that the Mia Hand follows the human closing motion, though with a distinct behavior compared to the Barrett Hand. The human hand performs a rapid closure starting at $t \approx 10.3$ s, reaching a minimum radius before relaxing slightly to a steady state of ≈ 3 cm. The Mia Hand responds with a gentler slope, indicating a smoother actuation response.

In particular, we can note how the robot continues to close until it reaches a final radius of ≈ 2.5 cm, which is smaller than the human’s final posture. It does not mimic the slight “relaxation” (radius increase) seen in the human trajectory at $t \approx 11.0$ s; instead, it maintains the tightest configuration it achieved, maximizing contact with the object.

Position variation The top-right graph highlights a very effective tracking of the workspace displacement. Unlike the previous case, the Mia Hand does not undershoot; in fact, it matches

and slightly exceeds the human displacement, settling at ≈ 4.4 cm compared to the human's ≈ 4.2 cm.

We can observe a "bump" during the transient of the robot's trajectory around $t \approx 10.5$ s. Since this experiment is performed in nominal conditions to test the retargeting algorithm on the *virtual* spheres, this behavior is not due to collision with a physical object. Looking at the simulation, the thumb reaches its limit during the closing motion quite early, forcing the hand to close the other fingers to compensate.

Elastic energy The energy plot (bottom) clearly reflects the kinematic behaviors described above. Due to the slower closing speed, the robot's energy accumulation lags behind the human reference during the transient phase (10.5 s – 11.0 s). However, as the robot achieves a tighter radius than the human, the curves cross at $t \approx 11.0$ s, with the robot stabilizing at a higher energy level (≈ 0.22 J) compared to the human (≈ 0.16 J), confirming that despite the initial delay, the final grasp is stable and exerts a higher virtual force on the object.

5.1.3 Shadow Dexterous Hand

The Shadow Dexterous Hand, being highly actuated and kinematically redundant for the grasping task, presents the closest approximation to the human hand among the three robotic hands considered. However, this high dimensionality also introduces greater complexity in the control loop.

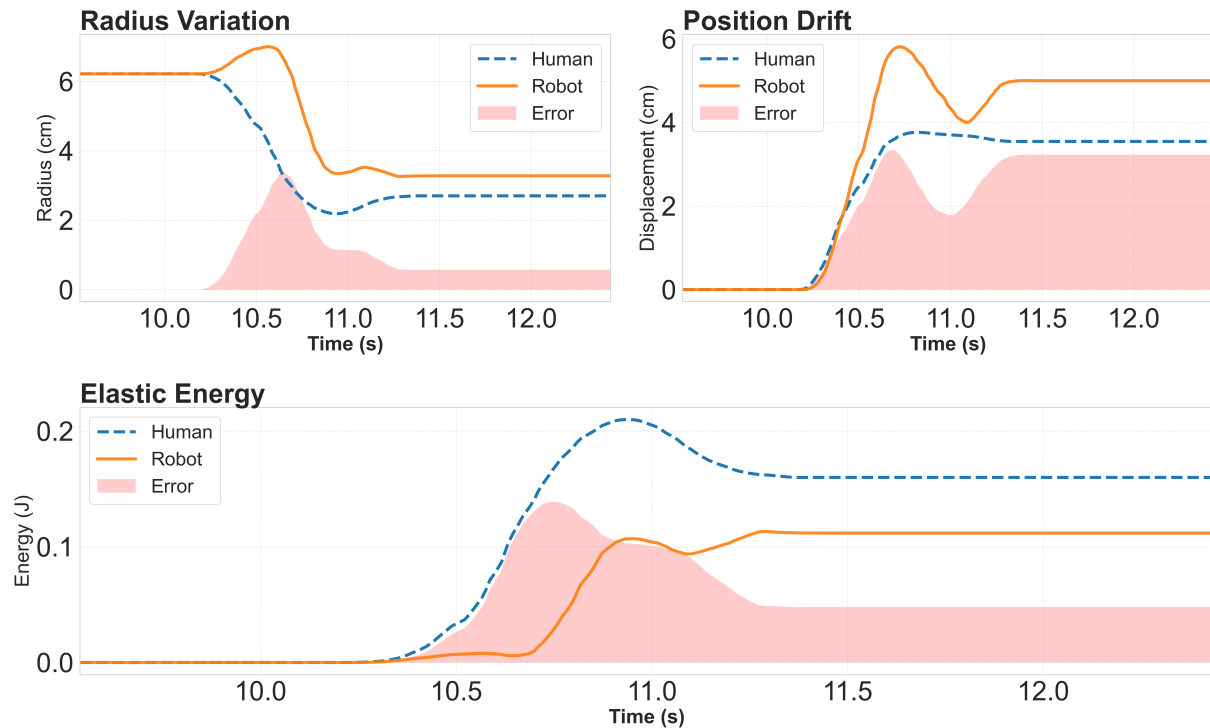


Figure 5.4: Performance metrics for the Shadow Dexterous Hand.

Radius comparison The radius tracking performance (top-left graph of Figure 5.4) shows a distinct behavior compared to the other robotic hands. While the human hand begins to close at $t \approx 10.3$ s, the Shadow Hand exhibits an initial "counter-movement", where the virtual sphere

radius actually *increases* from ≈ 6.2 cm to nearly 7 cm before the closing phase begins at $t \approx 10.7$ s.

This temporary divergence suggests that the inverse kinematics solver, constrained by the structure of the hand, found a path that required opening the fingers before converging on the target shrinking motion. Once the closing phase begins, the robot stabilizes at a final radius of ≈ 3.2 cm, which is slightly larger than the human's ≈ 2.8 cm.

Position variation The position tracking (top-right graph) further highlights the effect of this behavior. Unlike the conservative behavior observed in the previous hands, the Shadow hand shows a significant overshoot. At $t \approx 10.7$ s, the displacement spikes to ≈ 6 cm, that is nearly double the human reference, before settling back down to a steady-state displacement of ≈ 5 cm.

This oscillation is directly caused by the finger movement seen in the radius plot. When the robot extends its fingers to open the hand (between 10.3 s and 10.7 s), it pushes the center of the virtual sphere away from the palm, resulting in a large forward displacement. As the fingers close, they pull the sphere center back towards the palm, causing the displacement value to decrease and finally stabilize.

Elastic energy The energy profile (bottom) confirms these observations. We see a delay of almost 0.5 seconds before the energy starts to rise, matching the time the robot spent opening its hand instead of closing it. Furthermore, the Shadow Hand is the only robot to stabilize at a *lower* energy level (≈ 0.11 J) compared to the human (≈ 0.16 J). This is consistent with the radius plot: since the robot did not close its hand as tightly as the human, the resulting virtual grasp intensity is lower.

5.2 Impact of redundancy resolution

A key component of our control framework is the redundancy resolution strategy described in Chapter 3, which exploits the null space of the robot Jacobian to optimize the internal configuration of the robot. This additional optimization did not show significant effects on the performance metrics analyzed in the previous section, as they only consider a single closing motion. Therefore, the primary task, that is tracking the virtual sphere so as to perform a stable grasp, can be achieved without leveraging redundancy, even in the case of the dexterous Shadow Hand. However, our experiments highlighted how redundancy resolution plays a crucial role when it comes to maintaining a stable and natural behavior over time, when the user performs multiple grasps and releases in sequence.

5.3 Qualitative validation

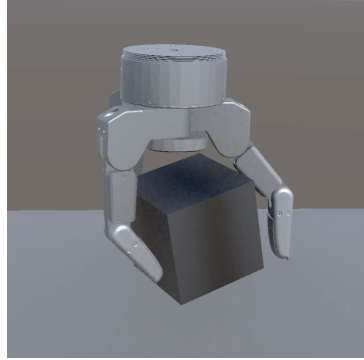
The various grasping tasks with each robotic hand were performed on a ball, a cube, and a cylinder. These shapes were chosen for their simplicity and their ability to represent common grasping scenarios. In the following we show representative images of each robotic hand successfully grasping the objects using the motion retargeting framework.

5.3.1 Barrett Hand

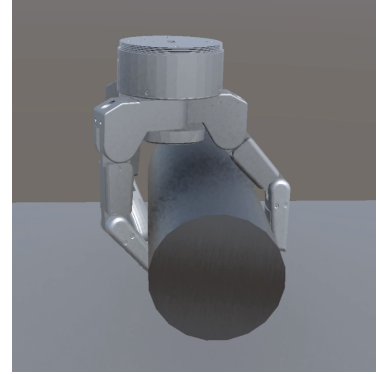
The Barrett Hand demonstrated robust performance across all three test objects. In Figure 5.5 we can see how it effectively conforms to the shape of each object, achieving secure grasps: these resulted from an initial configuration where the three fingers were open in a claw-like posture.



(a) Barrett Hand grasping a ball.



(b) Barrett Hand grasping a cube.



(c) Barrett Hand grasping a cylinder.

Figure 5.5: Barrett Hand grasping standard geometric objects.

5.3.2 Mia Hand

Figure 5.6 shows the Mia Hand successfully grasping the three objects from an initial open configuration. The anthropomorphic design and the underactuation of this hand allowed for a very simple yet effective grasping strategy, where the fingers naturally adapt to the shape of the objects upon collision.



(a) Mia Hand grasping a ball.



(b) Mia Hand grasping a cube.



(c) Mia Hand grasping a cylinder.

Figure 5.6: Mia Hand grasping standard geometric objects.

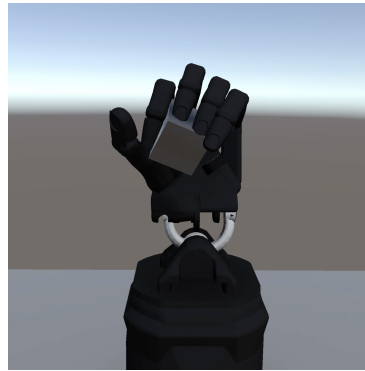
5.3.3 Shadow Hand

Finally, the Shadow Dexterous Hand leveraged its high dexterity to achieve natural, anthropomorphic grasps on all three objects. Unlike the trajectory challenges observed in free space, the

interaction with physical objects helped constrain the finger movements, resulting in more realistic configurations. As shown in Figure 5.7, the robot successfully used its multiple degrees of freedom to wrap its fingers closely around the ball, cube, and cylinder, resulting in stable power grasps that closely mimic human hand postures.



(a) Shadow Hand grasping a ball.



(b) Shadow Hand grasping a cube.



(c) Shadow Hand grasping a cylinder.

Figure 5.7: Shadow Hand grasping standard geometric objects.

Chapter 6

Conclusion

In this work, we developed a system to control robotic hands using the motion of a human hand. We used a method called the *virtual sphere*, which simplifies the problem by looking at how the hand interacts with an object rather than mapping the joints directly. We tested this system in a simulation with three different robotic hands: the Barrett Hand, the Mia Hand, and the Shadow Dexterous Hand.

6.1 Summary

Our experiments showed that the virtual sphere method is effective for controlling robotic hands with very different kinematic structures. Here is a summary of our main findings:

- The system successfully mapped the opening and closing of the human hand to all three robots, which managed to track the human radius changes well.
- All robots followed the direction of the human hand correctly, demonstrating the ability of the method to adapt to different initial configurations, which still affect the final grasping posture.
- The settings of the physical parameters of each robot, such as stiffness and damping, played a crucial role in the quality of the retargeted motion as well as the grasping success.
- The secondary objective of keeping the robot joints in their mid-range values did not have a significant impact on the single grasping tasks performed in this work. However, it showed a stabilizing effect on the robot posture in long sequences of motion, preventing extreme and unnatural joint configurations.
- All hands could grasp standard objects successfully, proving that the method works for different shapes.

6.2 Future work

There are several ways to improve and extend this work in the future:

1. So far, we have only tested the system in the Unity simulator. The next step is to connect real robotic hands to the system to see how friction, sensor noise, and communication delays affect performance in the real world.

2. The Weart TouchDIVER glove can produce forces and vibrations. Currently, communication goes only one way (from glove to Unity). We could send collision data from Unity back to the glove so the user can feel the object being grasped.
3. We used a sphere to represent the virtual object. Using different shapes, like an ellipsoid, might help grasp long or thin objects more accurately.

Bibliography

- [1] G. Gioioso, G. Salvietti, M. Malvezzi, and D. Prattichizzo, “Mapping synergies from human to robotic hands with dissimilar kinematics: An approach in the object domain,” *IEEE Transactions on Robotics*, vol. 29, no. 4, pp. 825–837, 2013.
- [2] M. Santello, M. Flanders, and J. F. Soechting, “Postural hand synergies for tool use,” *The Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998.
- [3] C. L. Primiceri, S. Trovalusci, D. I. Bubenek Turconi, and G. Pagano, *Motion retargeting on a human hand model*, M.Sc. Project Report, Sapienza University of Rome, Code available at: <https://github.com/serenatrovalusci/Hand-Motion-Reconstruction-from-Minimal-Inputs-through-Latent-Space-Learning>, Jun. 2025.
- [4] A. Fabisch, M. Uliano, D. Marschner, M. Laux, J. Brust, and M. Controzzi, “A modular approach to the embodiment of hand motions from human demonstrations,” in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, 2022, pp. 801–808. DOI: [10.1109/Humanoids53995.2022.10000165](https://doi.org/10.1109/Humanoids53995.2022.10000165).
- [5] R. Ding, Y. Qin, J. Zhu, *et al.*, “Bunny-visionpro: Real-time bimanual dexterous teleoperation for imitation learning,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.03162>.